

Mesure et évolution de la volatilité d'un indice financier : cas du S&P 500

Auteur : HAYANA Sébastien

Formation : Master 2 en Probabilités, Statistique et Finance

Établissement : Université de Yaoundé 1

Cameroun, 2026

Analyse de la volatilité d'un indice financier

Introduction

Ce projet porte sur l'évolution de la volatilité d'un indice financier à partir de ses prix historiques.

L'objectif est de calculer les rendements journaliers, de mesurer la volatilité sur différentes fenêtres et d'identifier les périodes calmes et agitées.

Lien avec le second projet

Dans ce projet, j'étudie l'évolution de la volatilité du S&P 500 au cours du temps. J'utilise une volatilité glissante sur 20 jours, puis une comparaison avec une fenêtre de 60 jours, ce qui me permettra d'observer les périodes où le marché est relativement plus calme ou plus agité.

Le projet 2 aborde ensuite la volatilité d'une manière plus simple, en la considérant comme constante sur la période étudiée. Cette hypothèse facilite les calculs, mais elle ne permet pas de suivre les changements du risque au fil du temps, ce qui permet de mieux capturer la dynamique réelle des marchés financiers.

Les deux projets sont complémentaires, mais ils ne répondent pas exactement à la même question. Le projet 1 montre que la volatilité peut évoluer selon les périodes, tandis que le projet 2 étudie le cas simplifié d'une volatilité fixe dans le cadre de cette étude.

Démarche de projet

J'ai réalisé ce projet progressivement. J'ai d'abord téléchargé les données du S&P 500, puis j'ai examiné leur structure avant de calculer les rendements journaliers et la volatilité glissante. J'ai ensuite comparé les fenêtres de 20 et 60 séances afin d'identifier les périodes plus calmes ou plus agitées.

Pour cette analyse, j'ai utilisé Python et les bibliothèques pandas, NumPy, Matplotlib et yfinance. Chaque étape a été vérifiée dans le notebook afin de mieux comprendre les calculs et d'interpréter les résultats obtenus.

```
In [ ]: # !pip install yfinance
```

```
In [1]: %pip install --upgrade --no-cache-dir yfinance
```

```
Requirement already satisfied: yfinance in c:\users\lenovo\anaconda3\lib\site-packages (1.7.0)
Requirement already satisfied: beautifulsoup4>=4.11.1 in c:\users\lenovo\anaconda3\lib\site-packages (from yfinance) (4.12.3)
Requirement already satisfied: curl_cffi>=0.15 in c:\users\lenovo\anaconda3\lib\site-packages (from yfinance) (0.16.3)
Requirement already satisfied: lxml>=4.9.0 in c:\users\lenovo\anaconda3\lib\site-packages (from yfinance) (5.2.1)
Requirement already satisfied: multitasking>=0.0.7 in c:\users\lenovo\anaconda3\lib\site-packages (from yfinance) (0.0.13)
Requirement already satisfied: numpy>=1.16.5 in c:\users\lenovo\anaconda3\lib\site-packages (from yfinance) (1.26.4)
Requirement already satisfied: pandas>=1.3.0 in c:\users\lenovo\anaconda3\lib\site-packages (from yfinance) (2.2.2)
Requirement already satisfied: peewee>=3.16.2 in c:\users\lenovo\anaconda3\lib\site-packages (from yfinance) (4.5.1)
Requirement already satisfied: platformdirs>=2.0.0 in c:\users\lenovo\anaconda3\lib\site-packages (from yfinance) (3.10.0)
Requirement already satisfied: protobuf>=3.19.0 in c:\users\lenovo\anaconda3\lib\site-packages (from yfinance) (3.20.3)
Requirement already satisfied: pytz>=2022.5 in c:\users\lenovo\anaconda3\lib\site-packages (from yfinance) (2024.1)
Requirement already satisfied: requests>=2.31 in c:\users\lenovo\anaconda3\lib\site-packages (from yfinance) (2.32.2)
Requirement already satisfied: websockets>=13.0 in c:\users\lenovo\anaconda3\lib\site-packages (from yfinance) (17.1)
Requirement already satisfied: soupsieve>1.2 in c:\users\lenovo\anaconda3\lib\site-packages (from beautifulsoup4>=4.11.1->yfinance) (2.5)
Requirement already satisfied: cffi>=2.0.0 in c:\users\lenovo\anaconda3\lib\site-packages (from curl_cffi>=0.15->yfinance) (2.1.1)
Requirement already satisfied: certifi>=2024.2.2 in c:\users\lenovo\anaconda3\lib\site-packages (from curl_cffi>=0.15->yfinance) (2024.6.2)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\lenovo\anaconda3\lib\site-packages (from pandas>=1.3.0->yfinance) (2.9.0.post0)
Requirement already satisfied: tzdata>=2022.7 in c:\users\lenovo\anaconda3\lib\site-packages (from pandas>=1.3.0->yfinance) (2023.3)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\lenovo\anaconda3\lib\site-packages (from requests>=2.31->yfinance) (2.0.4)
Requirement already satisfied: idna<4,>=2.5 in c:\users\lenovo\anaconda3\lib\site-packages (from requests>=2.31->yfinance) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\lenovo\anaconda3\lib\site-packages (from requests>=2.31->yfinance) (2.2.2)
Requirement already satisfied: pycparser in c:\users\lenovo\anaconda3\lib\site-packages (from cffi>=2.0.0->curl_cffi>=0.15->yfinance) (2.21)
Requirement already satisfied: six>=1.5 in c:\users\lenovo\anaconda3\lib\site-packages (from python-dateutil>=2.8.2->pandas>=1.3.0->yfinance) (1.16.0)
Note: you may need to restart the kernel to use updated packages.
```

```
In [2]: import yfinance as yf
print ("C'est bien là!")
```

C'est bien là!

```
In [3]: # import des librairies
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import yfinance as yf
```

```
In [4]: # choix de actif et téléchargement des données
ticker = "^GSPC" # S&P 500
data = yf.download(ticker,period="2y", interval="1d")
```

```
[*****100%*****] 1 of 1 completed
```

Exploration des données

Nous commençons par observer la structure du tableau téléchargé.

```
In [5]: print(data.head())
```

Price	Close	High	Low	Open	Volume
Ticker	^GSPC	^GSPC	^GSPC	^GSPC	^GSPC
Date					
2024-09-30	5762.479980	5765.140137	5703.529785	5726.520020	4425730000
2024-10-01	5708.750000	5757.729980	5681.279785	5757.729980	4025180000
2024-10-02	5709.540039	5719.629883	5674.000000	5698.140137	3829050000
2024-10-03	5699.939941	5718.779785	5677.370117	5698.189941	3597450000
2024-10-04	5751.069824	5753.209961	5702.830078	5737.479980	3479400000

```
In [6]: print(data.head(10))
```

Price	Close	High	Low	Open	Volume
Ticker	^GSPC	^GSPC	^GSPC	^GSPC	^GSPC
Date					
2024-09-30	5762.479980	5765.140137	5703.529785	5726.520020	4425730000
2024-10-01	5708.750000	5757.729980	5681.279785	5757.729980	4025180000
2024-10-02	5709.540039	5719.629883	5674.000000	5698.140137	3829050000
2024-10-03	5699.939941	5718.779785	5677.370117	5698.189941	3597450000
2024-10-04	5751.069824	5753.209961	5702.830078	5737.479980	3479400000
2024-10-07	5695.939941	5739.339844	5686.850098	5737.799805	3637790000
2024-10-08	5751.129883	5757.600098	5714.560059	5719.140137	3393400000
2024-10-09	5792.040039	5796.799805	5745.020020	5751.799805	3650340000
2024-10-10	5780.049805	5795.029785	5764.759766	5778.359863	3208790000
2024-10-11	5815.029785	5822.129883	5775.089844	5775.089844	3208720000

```
In [7]: print( "nombre de lignes:",len(data))
```

nombre de lignes: 502

```
In [8]: print(data.shape)
print (data.columns)
print(data.index)
```

```
(502, 5)
MultiIndex([( 'Close', '^GSPC'),
             ( 'High', '^GSPC'),
             (  'Low', '^GSPC'),
             ( 'Open', '^GSPC'),
             ('Volume', '^GSPC')],
            names=['Price', 'Ticker'])
DatetimeIndex(['2024-09-30', '2024-10-01', '2024-10-02', '2024-10-03',
               '2024-10-04', '2024-10-07', '2024-10-08', '2024-10-09',
               '2024-10-10', '2024-10-11',
               ...
               '2026-09-17', '2026-09-18', '2026-09-21', '2026-09-22',
               '2026-09-23', '2026-09-24', '2026-09-25', '2026-09-28',
               '2026-09-29', '2026-09-30'],
              dtype='datetime64[ns]', name='Date', length=502, freq=None)
```

```
In [9]: print(data.info())
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 502 entries, 2024-09-30 to 2026-09-30
Data columns (total 5 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   (Close, ^GSPC)        502 non-null   float64
 1   (High, ^GSPC)         502 non-null   float64
 2   (Low, ^GSPC)          502 non-null   float64
 3   (Open, ^GSPC)         502 non-null   float64
 4   (Volume, ^GSPC)       502 non-null   int64
dtypes: float64(4), int64(1)
memory usage: 23.5 KB
None
```

```
In [10]: close = data["Close"]
         print(close.head())
```

```
Ticker      ^GSPC
Date
2024-09-30  5762.479980
2024-10-01  5708.750000
2024-10-02  5709.540039
2024-10-03  5699.939941
2024-10-04  5751.069824
```

Évolution du prix de clôture

Le graphique suivant permet d'analyser la tendance générale du prix au cours de la période étudiée.

```
In [11]: import matplotlib.pyplot as plt

         close.plot(figsize=(10,4),title= "prix de clôture du S&P 500")
         plt.xlabel("Date")
         plt.ylabel("prix")
         plt.grid(True)
         plt.show()
```



Le prix évolue de manière variable au cours de la période donnée. Les fluctuations visibles ne mesurent pas directement la volatilité comme on pourrait le penser au début ; C'est pourquoi je calcule ensuite les rendements et leur écart type.

Méthodologie

Les prix de clôture ont été utilisés pour calculer les rendements journaliers . En effet , je m'intéresse aux paramètres qui résument la journée boursière.

Le rendement simple est calculé avec la formule ci-dessus qui mesure la variation relative entre deux prix successifs:

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

La volatilité sur 20 jours correspond à l'écart type des rendements observées sur une fenêtre mobile de 20 jours. La volatilité est ensuite annualisée en la multipliant par la racine carrée de 252 qui correspond approximativement aux séances de marché durant une année ou aux jours de cotation boursière dans une année.

Remarque sur le rendement logarithmique

Une autre mesure courante est le rendement logarithmique, défini par :

$$r_t^{log} = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

Il est également appelé rendement à capitalisation continue. Contrairement au rendement simple, les rendements logarithmiques peuvent être additionnés sur plusieurs périodes pour obtenir le rendement logarithmique cumulé.

Dans ce projet, le rendement simple a été retenu car il est plus intuitif et permet d'interpréter directement les variations quotidiennes en pourcentage. Le rendement logarithmique aurait toutefois été adapté à une analyse statistique ou à une modélisation plus avancée.

Calcul des rendements

Nous calculons les variations journalières du prix de clôture afin de mesurer les mouvements quotidiens de l'indice.

```
In [12]: # CALCUL DES RENDEMENTS SIMPLES
returns = close.pct_change() # La methode pct_change compare chaque valeur av
print(returns.head())
```

Ticker	^GSPC
Date	
2024-09-30	NaN
2024-10-01	-0.009324
2024-10-02	0.000138
2024-10-03	-0.001681
2024-10-04	0.008970

```
In [13]: data["return"] = data["Close"].pct_change() # La methode pct_change compare chaqu
# Là, la colonne des rendements journaliers a été créée .
```

```
In [14]: print(data[["Close", "return"]].head())
```

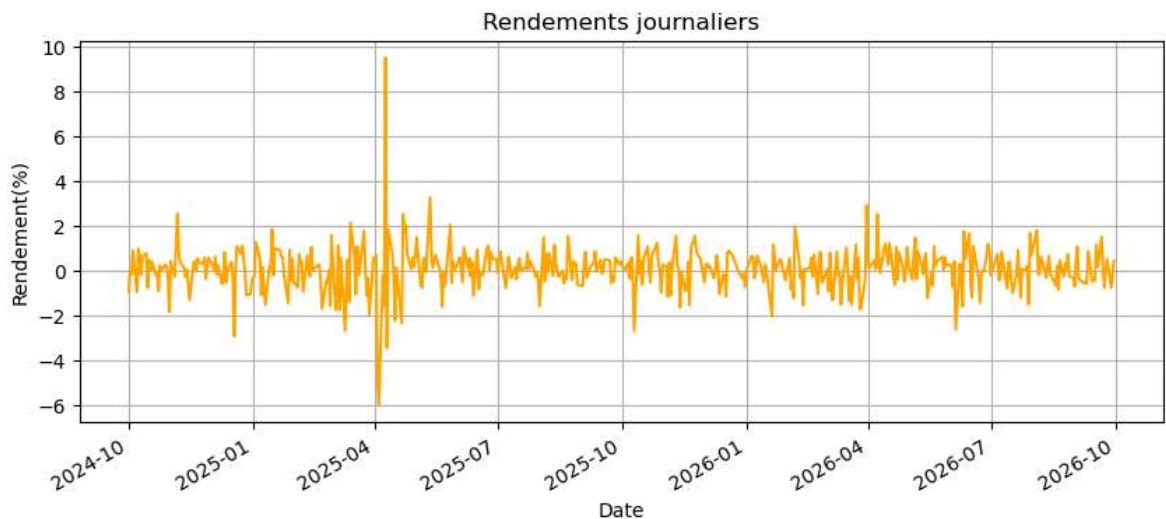
Price	Close	return
Ticker	^GSPC	
Date		
2024-09-30	5762.479980	NaN
2024-10-01	5708.750000	-0.009324
2024-10-02	5709.540039	0.000138
2024-10-03	5699.939941	-0.001681
2024-10-04	5751.069824	0.008970

La première valeur est **NaN**, car aucun rendement ne peut être calculé pour la première date : il n'existe en fait pas encore de prix précédent.

Évolution des rendements journaliers

```
In [15]: # tracé des rendements

(data["return"]*100).plot(figsize=(10,4),title= "Rendements journaliers",color=
plt.xlabel("Date")
plt.ylabel("Rendement(%)")
plt.grid(True)
plt.show()
```



Interprétation des rendements journaliers

Le graphique montre les variations quotidiennes du prix sous forme de rendements.

Les valeurs positives correspondent aux journées où le prix a augmenté, tandis que les valeurs négatives correspondent aux journées où le prix a diminué. Les rendements sont généralement proches de zéro, mais certaines dates présentent encore des mouvements plus importants.

Les pics positifs et négatifs indiquent des journées où le marché a connu des variations plus fortes que d'habitude. Ce graphique permet peut-être d'identifier des mouvements journaliers importants, mais il ne mesure pas directement la volatilité sur une période complète.

```
In [16]: print( data.shape)
print(data.columns)      # Petite vérification pour l'ajout de la colonne des rend
print(data[["Close","return"]].head())
```

```
(502, 6)
MultiIndex([( 'Close', '^GSPC'),
            ( 'High', '^GSPC'),
            ( 'Low', '^GSPC'),
            ( 'Open', '^GSPC'),
            ('Volume', '^GSPC'),
            ('return', '')],
            names=['Price', 'Ticker'])
Price      Close      return
Ticker      ^GSPC
Date
2024-09-30  5762.479980      NaN
2024-10-01  5708.750000 -0.009324
2024-10-02  5709.540039  0.000138
2024-10-03  5699.939941 -0.001681
2024-10-04  5751.069824  0.008970
```

Volatilité glissante sur 20 jours et graphique

Nous calculons la volatilité sur une fenêtre glissante de 20 jours. Pour chaque date, l'écart type est calculé à partir des rendements des 20 dernières séances.

Nous présentons ensuite cette volatilité sous deux formes:

*La volatilité non annualisée, qui reste exprimée sur l'échelle des rendements journaliers.

*La volatilité annualisée qui convertit notre mesure journalière en une estimation sur une base annuelle

La volatilité annualisée est obtenue en multipliant la volatilité non annualisée par la racine carrée de 252, qui correspond approximativement au nombre de séances boursières dans une année.

```
In [17]: data["volatility_20d"] = data["return"].rolling(window=20).std()
```

```
In [18]: data.columns = data.columns.get_level_values(0) # afin de faciliter la lecture
```

```
In [19]: print(data[["return", "volatility_20d"]].head(25)) # Afin de remarquer logiquement
```

Price	return	volatility_20d
Date		
2024-09-30	NaN	NaN
2024-10-01	-0.009324	NaN
2024-10-02	0.000138	NaN
2024-10-03	-0.001681	NaN
2024-10-04	0.008970	NaN
2024-10-07	-0.009586	NaN
2024-10-08	0.009689	NaN
2024-10-09	0.007113	NaN
2024-10-10	-0.002070	NaN
2024-10-11	0.006052	NaN
2024-10-14	0.007708	NaN
2024-10-15	-0.007609	NaN
2024-10-16	0.004679	NaN
2024-10-17	-0.000171	NaN
2024-10-18	0.003972	NaN
2024-10-21	-0.001823	NaN
2024-10-22	-0.000475	NaN
2024-10-23	-0.009191	NaN
2024-10-24	0.002146	NaN
2024-10-25	-0.000299	NaN
2024-10-28	0.002651	0.006054
2024-10-29	0.001614	0.005592
2024-10-30	-0.003300	0.005675
2024-10-31	-0.018615	0.007154
2024-11-01	0.004093	0.006914

```
In [20]: data["volatility_20d"].plot(figsize=(10,4), title="volatilité glissante sur 20 jours")
plt.xlabel("Date")
plt.ylabel("Ecart type des rendements")
plt.grid(True)
plt.show()
```



Interprétation

La courbe représente la variation de la dispersion des rendements au fil du temps.

Lorsque la courbe augmente, les rendements des 20 dernières séances sont davantage dispersés. Le marché a alors connu des mouvements plus irréguliers ou plus importants.

Lorsque la courbe diminue, les rendements récents sont plus proches les uns des autres et la période est relativement plus calme.

Cette mesure est utile pour observer l'évolution de la volatilité dans le temps. Toutefois, comme elle n'est pas annualisée, ses valeurs sont moins faciles à comparer avec des indicateurs financiers généralement exprimés en pourcentage annuel. Ainsi, cette volatilité sera exprimée en pourcentage annuel dans la suite.

```
In [21]: print(data[["return", "volatility_20d"]].tail()) # Curieux de savoir à quoi resse
print(data["volatility_20d"].dropna().iloc[-1]) # iloc affiche la dernière valeur
```

Price	return	volatility_20d
2026-09-24	-0.000247	0.006911
2026-09-25	0.005099	0.006814
2026-09-28	-0.007712	0.007016
2026-09-29	-0.001672	0.006986
2026-09-30	0.004267	0.006845

Volatilité annualisée et graphique

Je convertis maintenant la volatilité non annualisée en volatilité annualisée. Ce redimensionnement nous permettra d'avoir une volatilité sur une base annuelle et exprimée en pourcentage.

```
In [22]: data["volatility_20d_annualized"] = (data["volatility_20d"] * (252 ** 0.5))
```

```
In [23]: data["volatility_20d_annualized_pct"] = (data["volatility_20d_annualized"] * 100)
```

```
In [24]: print(data[["return", "volatility_20d", "volatility_20d_annualized_pct"]].tail())
```

Price	return	volatility_20d	volatility_20d_annualized_pct
Date			
2026-09-24	-0.000247	0.006911	10.971050
2026-09-25	0.005099	0.006814	10.817412
2026-09-28	-0.007712	0.007016	11.137570
2026-09-29	-0.001672	0.006986	11.090724
2026-09-30	0.004267	0.006845	10.866506

Ce graphique ci dessous reprend la même volatilité calculée sur une fenêtre de 20 séances , mais annualisée et exprimée en pourcentage.

```
In [25]: data["volatility_20d_annualized_pct"].plot( figsize=(10,4),title= "volatiltité a
plt.xlabel("Date")
plt.ylabel("volatilité annualiséé (%)")
plt.grid(True)
plt.show()
```



Interprétation

La volatilité annualisée présente une forme générale assez similaire à celle de de la volatilité non annualisée. Cela est naturel car les deux volatilités utilisent les mêmes rendements.

La différence réside uniquement au niveau de l'échelle où la volatilité annualisée est calculé en pourcentage annuel afin de faciliter son inteprétation et sa comparaison avec les indicateurs financiers.

Synthèse

Comparaison des deux mesures

La volatilité non annualisée et la volatilité annualisée sont calculées sur la même fenêtre glissante de 20 séances. La première conserve l'échelle issue des rendements journaliers, tandis que la seconde est convertie sur une base annuelle et exprimée en pourcentage.

L'annualisation ne modifie pas la dynamique de la courbe. Elle modifie seulement l'unité et l'échelle des valeurs. Les deux graphiques présentent donc généralement une forme

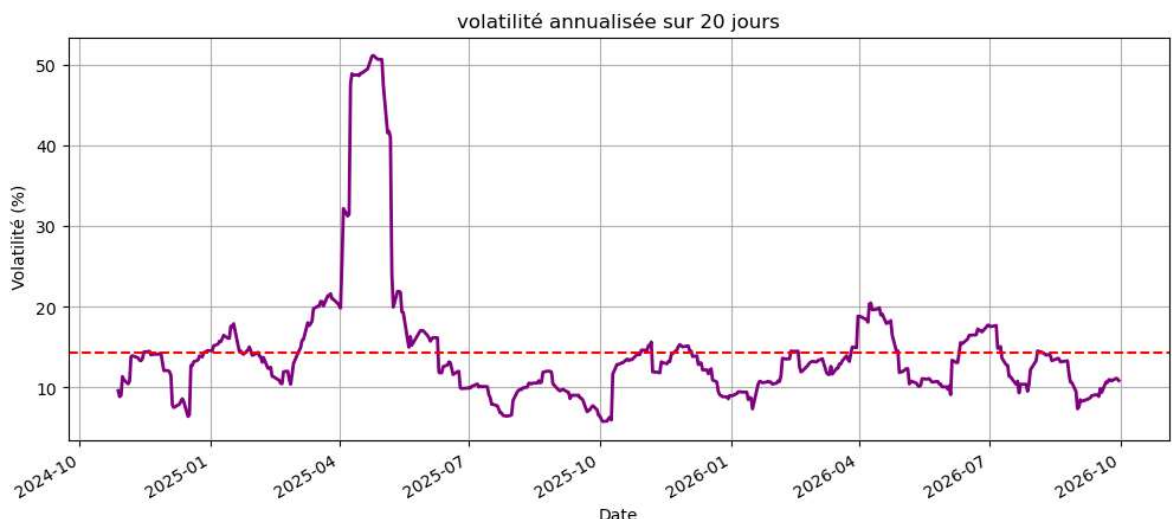
similaire, mais la volatilité annualisée est plus facile à interpréter dans un contexte financier.

Comparaison avec la volatilité moyenne

Je compare la volatilité annualisée sur une fenêtre de 20 jours à sa moyenne calculée sur toute la période étudiée. La moyenne sert de seuil de référence. Elle permet de distinguer les périodes où la volatilité est supérieure à son niveau habituel dans l'échantillon, et les périodes où elle inférieure.

Illustration

```
In [26]: vol = data["volatility_20d_annualized_pct"].dropna()
moyenne= vol.mean()
vol.plot(figsize= (12,5), color= "purple", linewidth= 2)
plt.axhline(moyenne,color= "red",linestyle="--")
plt.title("volatilité annualisée sur 20 jours")
plt.xlabel("Date")
plt.ylabel(" Volatilité (%)")
plt.grid(True)
plt.show()
```



Interprétation

La ligne rouge représente la volatilité moyenne calculée sur l'ensemble de la période étudiée.

La courbe violette représente la volatilité calculée sur les 20 dernières séances à chaque date.

Lorsque la courbe violette se situe au dessus de la ligne rouge, la volatilité sur les 20 dernières séances est supérieure à la moyenne de l'échantillon . Cela correspond à une période relativement plus agitée du marché.

Par contre,lorsque la courbe se situe en dessous de la ligne rouge , la volatilité sur les 20 dernières séances est inférieure à la moyenne.Cela correspond à une période relativement plus calme.

Notons que la dernière valeur de la courbe correspond à la volatilité calculée sur les 20 dernières séances disponibles et non le rendement du dernier jour uniquement.

Résumé statistique de la volatilité

Après avoir observé l'évolution de la volatilité sur le graphique , Je calcule à présent quatre indicateurs qui permettront de mesurer le degré de volatilité observée durant notre période étudiée: volatilité minimale,volatilité maximale,volatilité moyenne,volatilité récente.

```
In [27]: vol = data["volatility_20d_annualized_pct"].dropna() # On retire les valeurs ma
print("volatilité minimale:",round(vol.min(),2),"%")
print("volatilité maximale:",round(vol.max(),2),"%")
print("volatilité récente:",round(vol.iloc[-1],2),"%")
print("Volatilité moyenne:",round(vol.mean(),2),"%")
```

```
volatilité minimale: 5.81 %
volatilité maximale: 51.12 %
volatilité récente: 10.87 %
Volatilité moyenne: 14.32 %
```

Interprétation et synthèse

La volatilité minimale correspond à la période la plus faible observé pendant notre période étudiée.Elle correspond à une période où les rendements des 20 dernières séances étaient relativement très peu dispersés

La volatilité maximale correspond à la période la forte observée pendant notre période étudiée. Elle correspond à une période où les rendements des 20 jours étaient les plus dispersés pour notre période et où les mouvements de marché étaient importants .

La volatilité moyenne sert ici de seuil de référence pour l'ensemble de la période.

La volatilité récente ou encore la dernière valeur disponible de la volatilité résume les 20 dernières valeurs de rendements à la fin de l'échantillon. Ce qui est intéressant, c'est qu'elle peut être comparée à la moyenne pour déterminer si la période finale est relativement calme ou agitée : Dans notre cas , on voit très bien que la période est calme car la volatilité moyenne est de 14,32 % et la volatilité récente est de 10,87 % .

```
In [28]: # Les valeurs précédentes montre que la période actuelle du marché est plus calm
# si la récente était supérieure à la moyenne alors la periode actuelle serait p
#Si elles sont proches , l'utilisation actuelle est relativement normale
ecart= vol.iloc[-1] - vol.mean()
print(" écart par rapport à la moyenne:", round(ecart,2),"points de pourcentage
```

```
écart par rapport à la moyenne: -3.46 points de pourcentage
```

Comparaison avec la volatilité sur 60 jours et graphique

Je vais maintenant effectuer une comparaison, disons stratégique, de la volatilité sur différentes fenêtres notamment les fenêtres glissantes sur 20 jours et sur 60 jours.

Nous verrons que la volatilité glissante sur 20 jours réagit plus rapidement aux changements récents, autrement dit, le changement sera bien plus visible. Par contre, étant donné que la fenêtre de 60 jours utilise davantage d'observations, on aura généralement un aspect plus progressif de la volatilité et sera relativement moins visible.

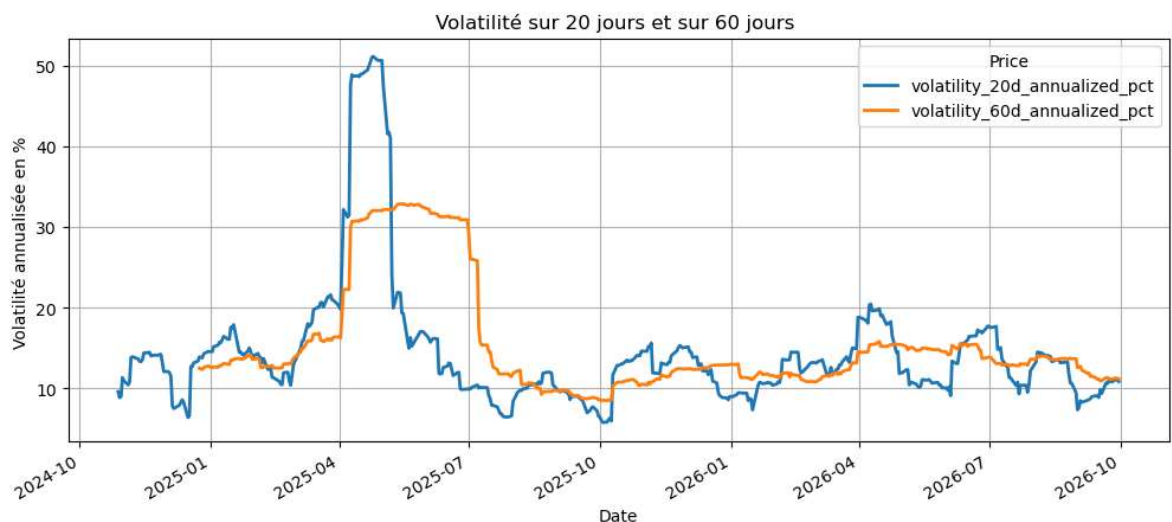
Les deux mesures sont annualisées et exprimées en pourcentage afin d'être sur le même d'échelle et d'être comparables.

```
In [29]: # Nous allons maintenant créer une variante sur 60 jours et faire une comparais

data["volatility_60d"] = (data["return"].rolling(window=60).std())
data["volatility_60d_annualized_pct"] = data["volatility_60d"] * (252**0.5) * 100
print(data[["volatility_20d_annualized_pct", "volatility_60d_annualized_pct"]].ta
```

Price	volatility_20d_annualized_pct	volatility_60d_annualized_pct
Date		
2026-09-24	10.971050	11.149026
2026-09-25	10.817412	11.175107
2026-09-28	11.137570	11.303657
2026-09-29	11.090724	11.222757
2026-09-30	10.866506	11.206686

```
In [30]: data[["volatility_20d_annualized_pct", "volatility_60d_annualized_pct"]].plot(figsize=(10, 6))
plt.xlabel("Date")
plt.ylabel("Volatilité annualisée en %")
plt.grid(True)
plt.show()
```



Remarque

La fenêtre de 60 jours a besoin de 60 rendements avant de calculer sa première valeur. Les premières lignes peuvent donc contenir la valeur NaN. C'est la raison pour laquelle les deux courbes démarrent tardivement et la courbe de la volatilité sur 60 jours démarre plus tardivement que la courbe de la volatilité sur 20 jours.

Interprétation

La volatilité sur 20 jours réagit plus rapidement aux changements récents que la volatilité sur 60 jours. C'est même d'ailleurs la raison pour laquelle elle présente des pics plus visibles. La volatilité sur 60 jours par contre réagit de façon plus progressive aux changements car elle prend en compte un grand nombre d'observations, c'est pourquoi on observe une courbe plus lisse.

Lorsque la volatilité sur 20 jours est au dessus de la volatilité sur 60 jours, cela signifie que la volatilité récente ou encore les mouvements récents sont plus importants que la tendance sur une longue période. En revanche, si la courbe passe en dessous de la courbe, cela signifie que les mouvements récents sont devenus relativement plus calmes que ceux observés sur les 60 dernières séances

Certes, nos deux courbes mesurent la volatilité annualisée, mais elles ne répondent pas à la même question : Les 20 jours décrivent le court terme tandis que les 60 jours décrivent une période plus progressive.

Corrélation et examen de la relation entre les rendements et la volatilité

Relation entre les rendements et la volatilité

J'étudie la relation entre les rendements journaliers et la volatilité annualisée sur 20 jours.

Deux analyses sont réalisées avec le même nuage de points. Dans la première, je conserve le signe du rendement afin d'étudier le lien avec la direction du mouvement. Dans la seconde, j'utilise la valeur absolue du rendement afin d'étudier le lien avec l'amplitude du mouvement, qu'il soit positif ou négatif.

La volatilité utilisée sur l'axe vertical est la volatilité annualisée sur 20 jours.

```
In [31]: # Analyse avec le rendement signé
analyse_signe = data[["return", "volatility_20d_annualized_pct"]].dropna()

correlation_signe = analyse_signe["return"].corr(analyse_signe["volatility_20d_a

# Analyse avec l'amplitude absolue du rendement
data["abs_return"] = data["return"].abs()

analyse_abs = data[["abs_return", "volatility_20d_annualized_pct"]].dropna()

correlation_abs = analyse_abs["abs_return"].corr(analyse_abs["volatility_20d_ann

print("Corrélation avec le rendement signé :", round(correlation_signe, 3))

print("Corrélation avec l'amplitude absolue :", round(correlation_abs, 3))
```

Corrélation avec le rendement signé : 0.081

Corrélation avec l'amplitude absolue : 0.354

Illustration

```
In [32]: analyse_abs.plot(
    x="abs_return",

    y="volatility_20d_annualized_pct",

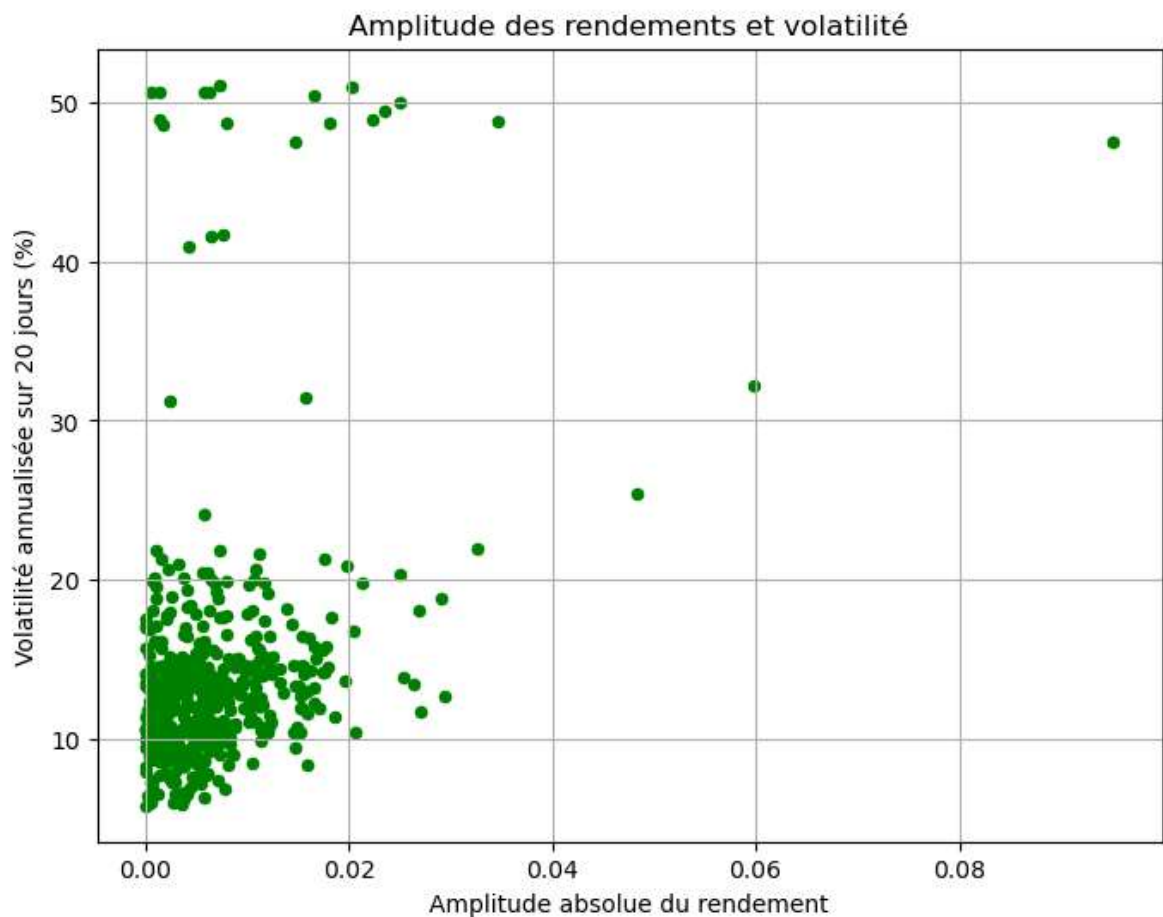
    kind="scatter",

    figsize=(8, 6),

    color= "green",

    title="Amplitude des rendements et volatilité"
)

plt.xlabel("Amplitude absolue du rendement")
plt.ylabel("Volatilité annualisée sur 20 jours (%)")
plt.grid(True)
plt.show()
```



Interprétation du nuage de points

Le nuage de points est assez dispersé et ne présente pas de tendance linéaire très nette.

La corrélation entre le rendement signé et la volatilité est de 0,081. Cette valeur, très proche de zéro, indique que la direction du rendement journalier explique très peu le niveau de volatilité récente.

Lorsque le rendement est remplacé par sa valeur absolue, la corrélation augmente à 0,354. Cette relation positive, faible à modérée, suggère que les mouvements journaliers de grande amplitude sont plus souvent associés à une volatilité élevée.

La différence entre les deux corrélations montre que la volatilité dépend davantage de l'importance des mouvements que de leur direction. Autrement dit, la volatilité est liée à l'ampleur des variations, qu'elles soient positives ou négatives. Toutefois, la dispersion du nuage de points indique que cette relation reste imparfaite.

Enfin, l'amplitude d'un seul rendement ne suffit pas à expliquer toute la volatilité calculée sur une fenêtre de 20 jours, car cette dernière dépend de l'ensemble des rendements observés pendant cette période.

Regime de volatilité

Classification des régimes de volatilité sur 20 jours

Je classe chaque date selon le niveau de volatilité annualisée sur 20 jours.

Une date est classée dans le régime de volatilité **élevée** lorsque sa volatilité est supérieure ou égale à la moyenne de la période (seuil). Elle est classée dans le régime de volatilité **faible** lorsque la volatilité est inférieure ou égale à la moyenne de la période.

Cette classification permet de repérer plus facilement les périodes relativement plus calmes et les périodes relativement plus agitées.

```
In [33]: seuil= vol.mean()

data["regime volatilite"] = "faible" #là je crée une nouvelle colonne où toutes
# on considère toutes les dates comme des périodes de volatilité faible

data.loc[ data["volatility_20d_annualized_pct"]> seuil,"regime volatilite"] = "e
print(data[["volatility_20d_annualized_pct","regime volatilite"]].tail())
```

Price	volatility_20d_annualized_pct	regime volatilite
Date		
2026-09-24	10.971050	faible
2026-09-25	10.817412	faible
2026-09-28	11.137570	faible
2026-09-29	11.090724	faible
2026-09-30	10.866506	faible

Remarque

Les dernières valeurs affichés sont au régime *faible* car les dernières valeurs sont inférieures au seuil moyen.

```
In [34]: print(data["regime volatilite"].value_counts())
```

```
regime volatilité
faible      351
élevée      151
Name: count, dtype: int64
```

Remarque

Il faut faire attention au calcul du nombre d'occurrences du régime **faible**. En effet les valeurs manquantes ou indisponibles de notre série de la volatilité annualisée sur 20 jours ont été considérées par défaut inférieures au seuil et prennent automatiquement la chaîne de caractère "faible". On règle ce problème avec la solution ci-dessous :

```
In [35]: import pandas as pd

def classer_regime(x):
    if pd.isna(x):
        return "indisponible"
    elif x > seuil:
        return "élevée"
    else:
        return "faible"

data["regime_volatilité"] = data["volatility_20d_annualized_pct"].apply(classer_
```

```
In [36]: print(data["regime_volatilité"].value_counts()) # Ceci permettra de régler le c

regime_volatilité
faible      331
élevée      151
indisponible  20
Name: count, dtype: int64
```

Remarque

On constate maintenant que le nombre d'occurrences du régime **faible** a diminué. Ceci confirme bien évidemment notre hypothèse passée.

Maximum et minimum

Je recherche maintenant les dates auxquelles la volatilité annualisée sur 20 jours a atteint sa valeur maximale ou sa valeur minimale.

```
In [37]: date_max = data["volatility_20d_annualized_pct"].idxmax()
valeur_max = data["volatility_20d_annualized_pct"].max()
print("Date du maximum:", date_max)
print("volatilité maximale:", round(valeur_max, 2), "%")
```

```
Date du maximum: 2025-04-25 00:00:00
volatilité maximale: 51.12 %
```

```
In [38]: print(data.loc[date_max]) # sélectionne toute la ligne à partir de l'étiquette re
# Curieux de voir tous les infos précises de cette date...
```

```

Price
Close                    5525.209961
High                    5528.109863
Low                     5455.859863
Open                    5489.72998
Volume                  4236580000
return                  0.007373
volatility_20d          0.032201
volatility_20d_annualized 0.511177
volatility_20d_annualized_pct 51.117717
volatility_60d          0.020189
volatility_60d_annualized_pct 32.049483
abs_return              0.007373
regime_volatilite       élevée
regime_volatilite       élevée
Name: 2025-04-25 00:00:00, dtype: object

```

```

In [39]: date_min = vol.idxmin()
        valeur_min = vol.min()

        print("Date du minimum:", date_min)
        print("volatilité minimale:", round(valeur_min, 2), "%")

```

```

Date du minimum: 2025-10-03 00:00:00
volatilité minimale: 5.81 %

```

Interprétation

La volatilité annualisée maximale est de 51,12 % le 25 avril 2025. Cette date correspond à la période la plus agitée de notre échantillon.

La volatilité minimale observée est de 5,81 % le 3 octobre 2025. Cette date correspond à la période la plus calme de l'échantillon.

Ces valeurs ne correspondent en aucun cas une perte ou un gain du marché mais plutôt des mesures de l'intensité fluctuations des rendements sur une fenêtre glissante de 20 jours exprimées en base annuelle.

Volatilité moyenne par régime

Cette étape consiste à vérifier le niveau de volatilité des régimes **faible** et **élevée**. Le groupe **indisponible** n'a pas de volatilité calculable car elle correspond justement aux premières dates où la fenêtre de 20 jours n'était pas encore complète.

```

In [40]: # La question suivante qu'on se pose: EST CE QUE LES RENDEMENTS MOYENS SONT DIF
        moyennes_regimes=data.groupby( "regime_volatilite")["return"].mean() # Là on es
        print("Rendements moyens par regime(%)")
        print(moyennes_regimes*100)

```

```

Rendements moyens par regime(%):
regime_volatilite
faible           0.034481
indisponible     0.043351
élevée           0.128245
Name: return, dtype: float64

```

Interprétation

La volatilité moyenne du régime élevée est supérieure à celle du régime faible, ce qui est cohérent et normal d'après notre règle de classification.

Le régime faible regroupe ici les dates où la volatilité est inférieure à la moyenne, alors que le régime élevée regroupe ceux dont la volatilité est supérieure à la volatilité moyenne.

Tableau récapitulatif

Le tableau suivant rassemble les principaux indicateurs calculés pendant l'analyse.

```
In [41]: resume = pd.DataFrame({ "indicateur": ["volatilité minimale (%)","volatilité max  
print(resume)
```

	indicateur	valeur
0	volatilité minimale (%)	5.810
1	volatilité maximale(%)	51.120
2	volatilité récente(%)	10.870
3	volatilité moyenne(%)	14.320
4	Corrélation amplitude-volatilité	0.354

La volatilité maximale est nettement supérieure à la volatilité minimale, ce qui montre que le niveau d'agitation du marché a fortement varié pendant notre période étudiée.

```
In [42]: print(data.columns) # Juste curieux de voir mon tableau de fin
```

```
Index(['Close', 'High', 'Low', 'Open', 'Volume', 'return', 'volatility_20d',  
      'volatility_20d_annualized', 'volatility_20d_annualized_pct',  
      'volatility_60d', 'volatility_60d_annualized_pct', 'abs_return',  
      'regime_volatilite', 'regime_volatilite'],  
      dtype='object', name='Price')
```

Résultats principaux

-La volatilité minimale observée est de 5,81%, le 3 octobre 2025.

-La volatilité maximale observée est de 51,12%, le 25 avril 2025.

-La volatilité récente est 10,87%.

-La corrélation entre l'amplitude absolue des rendements et la volatilité est de 0,354.

-La période étudiée contient 331 observations classées dans le régime de volatilité faible et 151 observations classées dans le régime de volatilité élevée.

-Enfin, 20 observations sont indisponibles au début de la série, car la fenêtre de 20 jours n'est pas encore complète.

Conclusion

Cette étude m'a permis d'analyser la volatilité d'un indice financier à partir de ses prix historiques.

Les rendements journaliers ont été calculés à partir de prix de clôture, puis une volatilité glissante sur 20 jours a été estimée et annualisée. Ensuite une comparaison avec une fenêtre de 60 jours a permis de distinguer les variations rapides des tendances plus générales.

La volatilité maximale observée est de 51,12 % le 25 avril 2025, tandis que la volatilité minimale est de 5,81 % le 3 octobre 2025. La volatilité récente, proche de 11 %, est nettement inférieure au maximum observée.

Enfin, la corrélation de 0,354 entre l'amplitude absolue des rendements et la volatilité indique une relation positive faible à modérée.

Ces résultats décrivent uniquement la période étudiée, mais ne permettent de prédire avec certitude les mouvements futurs du marché.

Après les calculs et les interprétations, nous enregistrons à présent le tableau final dans un fichier **CSV** afin de conserver les données pour les utiliser ultérieurement. Ce fichier contient les prix, les rendements, les volatilités sur 20 et 60 jours, les versions annualisées et la classification de nos régimes de volatilité.

```
In [43]: data.to_csv("analyse_volatilite_finale.csv", index=True)
```

```
In [44]: import os
print(os.path.exists("analyse_volatilite_finale.csv"))
```

True

Références

Yahoo Finance, données historiques du S&P 500 (^GSPC), utilisées comme source des prix de clôture : <https://finance.yahoo.com/quote/%5EGSPC/history/>

yfinance, documentation de la fonction `download()`, utilisée pour télécharger les données avec Python :

<https://ranaroussi.github.io/yfinance/reference/api/yfinance.download.html>

pandas, documentation de `pct_change()`, utilisée pour calculer les rendements journaliers : https://pandas.pydata.org/docs/reference/api/pandas.Series.pct_change.html

pandas, documentation de `rolling()`, utilisée pour calculer la volatilité sur une fenêtre mobile : <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.rolling.html>

```
In [ ]:
```